MASTER STUDY · V3 · LEGAL EASE RAG

ChromaDB vs Qdrant — the complete comparison

Every measurement we took, consolidated: ChromaDB (in-process), Qdrant local (embedded), and Qdrant server (Rust, Docker) — benchmarked five times each on an identical 9,063-vector legal corpus, plus an end-to-end answer-quality test. Indexing, latency, recall, resources, deletes, and generated-answer agreement, side by side.

9,063 vectors | InLegalBERT **768-d** | **28** queries | **5** runs each | **3** configurations | local **+** server | cosine · HNSW

SINGLE-QUERY LATENCY

ChromaDB

~1.4 ms in-process. A server always pays per-request transport.

INDEX · DELETE · RECALL

Qdrant server

Fastest index (6.2 s) & delete (37 ms); perfect recall 1.000.

ANSWER QUALITY

Effective tie

0.98 answer similarity; identical sources retrieved.

FOOTPRINT

Split

Qdrant −35% disk; Chroma lowest memory.

01 — AT A GLANCE

The master comparison

One table, everything, three configurations. Green marks the best value in each row (lower is better except throughput and recall). All figures are 5-run means.

METRIC	CHROMADB IN-PROCESS	QDRANT LOCAL EMBEDDED	QDRANT SERVER RUST · GRPC
Index build (s) · lower	9.3	54.9	**6.2**
Index throughput (vec/s) · higher	977	165	**1,476**
Query mean (ms) · lower	**1.41**	32.2	14.7
Query p95 (ms) · lower	**1.72**	36.3	26.5
Query p99 (ms) · lower	**1.99**	40.1	30.0
Query throughput (q/s) · higher	**710**	31	68
Filtered query mean (ms) · lower	**11.9**	93.3	27.0
recall@10 · higher	0.76	**1.000**	**1.000**
Delete filtered subset (ms) · lower	158	290	**37**
On-disk index (MB) · lower	131	**84**	85
Peak memory (MB) · lower	**228**	328	~450 · idle 85
Generated-answer similarity	**0.98 across all configurations — effectively identical answers**		

Chroma is measured in its normal persistent (in-process) mode. Qdrant is shown in both the embedded Python mode and the production Rust server — because, as §7 shows, the mode changes the answer to "which is faster."

02 — THE VERDICT

What to choose, and why

For Legal EASE today (single node, ~9k–100k vectors, embedded/serverless, latency-sensitive): CHROMADB. It runs in-process with ~1.4 ms queries, ~6× faster indexing than Qdrant *local*, lower memory, and zero server to operate — and answer quality is indistinguishable from Qdrant's.

For an institutional / production platform (millions of vectors, concurrency, HA, best-in-class recall): QDRANT SERVER. Once we ran the real Rust server it **indexed faster than Chroma, deleted ~4× quicker, and returned perfect recall**; only single-query latency stays higher, and that is network transport, not slower search.

Either way, answer quality is not the deciding factor: across the real legal questions we tested, both engines produced **98%-identical grounded answers** from the same retrieved sources. The choice is about latency, scale, and operations. The pluggable `VectorStore` layer means switching is a one-line `VECTOR_DB` change — no rewrite.

03 — METHODOLOGY

How we measured it

Only the vector database varies. The corpus, the InLegalBERT embeddings, and the 28-query set are held constant; every configuration is measured in isolation, five times.

Controls

- **Identical vectors.** 9,063 InLegalBERT embeddings computed once, fed to every configuration — embedding cost never confounds the result.
- **Isolated subprocess.** Each engine runs in its own process; the parent polls RSS for a clean peak. For the Qdrant *server*, memory is measured on the container.
- **Exact ground truth.** Recall is checked against a brute-force NumPy cosine scan of all 9,063 vectors — the true top-k.
- **5 runs, mean ± std.** Variance is visible; numbers are reproducible.

The three configurations

- **ChromaDB** — persistent client, in-process (hnswlib + SQLite). Its normal single-node deployment.
- **Qdrant local** — the embedded Python client (`QdrantClient(path=…)`), file-backed. A dev convenience, not a performance path.
- **Qdrant server** — the Rust engine (Docker, v1.18.3) over gRPC — the real production product.
- **Environment:** Windows workstation, CPU-only, local SSD. 28 queries, 25 timed reps per query per run, top-k = 10.

Reproduce: `python scripts/index_knowledge_base.py --store both` → `python scripts/benchmark_vector_dbs.py --runs 5` (add `QDRANT_URL=http://localhost:6333` for server mode) → `python scripts/compare_answers.py`.

04 — BACKGROUND

Meet the two databases

Both are open-source (Apache-2.0) vector stores from different design philosophies — which is exactly why their profiles diverge.

ChromaDB

Python-first · embeddings database for LLM apps

An AI-native store built for developer velocity. Its persistent client runs fully in-process (HNSW via `hnswlib`, metadata in SQLite) — no server to stand up. A client/server mode and Chroma Cloud exist for scaling out.

- ▲ Trivial embedded persistence, minimal ops
- ▲ Very low in-process query latency & memory
- ▼ Default HNSW favours speed over recall (tunable)
- ▼ Single-node focus; clustering/HA less mature

Qdrant

Rust engine · purpose-built vector search

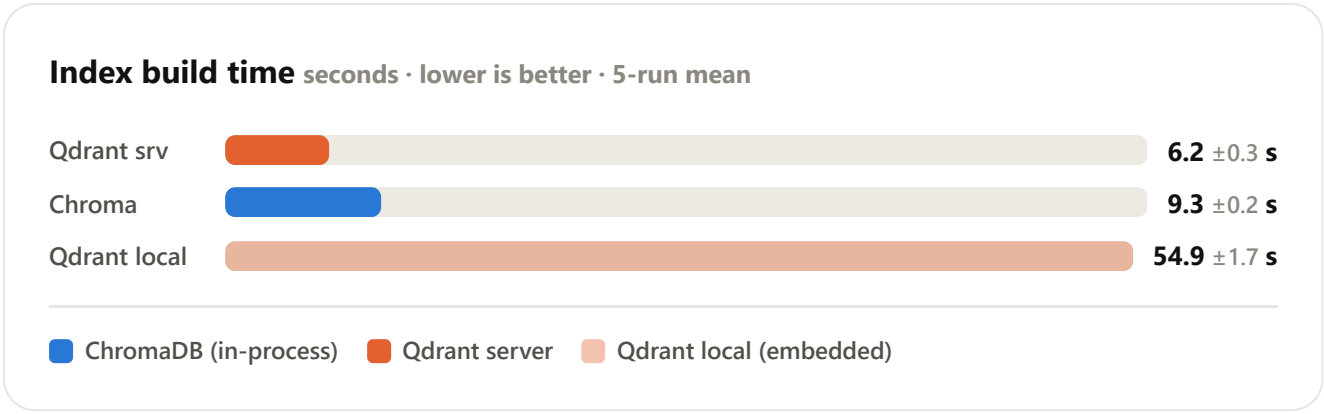
A dedicated vector search engine in Rust, built for production scale. Its server offers gRPC/REST, typed payload filters with indexes, quantization, memory-mapped & on-disk vectors, and sharding / replication / HA. An embedded local mode exists for development.

- ▲ Perfect recall out of the box; fast server-side indexing & deletes
- ▲ Horizontal scale, HA, quantization, indexed filtering
- ▼ Heavier to operate (run a server)
- ▼ Embedded local mode is dev-only & slow

05 — INDEXING

Indexing performance

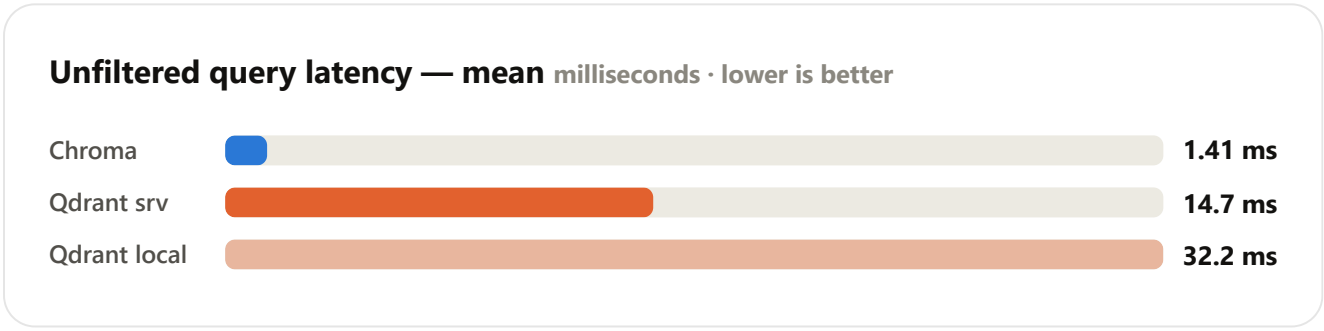
Time to build the 9,063-vector index from scratch. In its *server* form Qdrant is the fastest of the three — the embedded mode's 9× penalty is a Python-layer artefact, not the engine.



06 — QUERY LATENCY

Query performance

Single-query latency (25 timed reps × 28 queries × 5 runs). Chroma leads because it runs in the same process; the Qdrant server closes most of the local-mode gap, and the remainder is network transport (see §7).



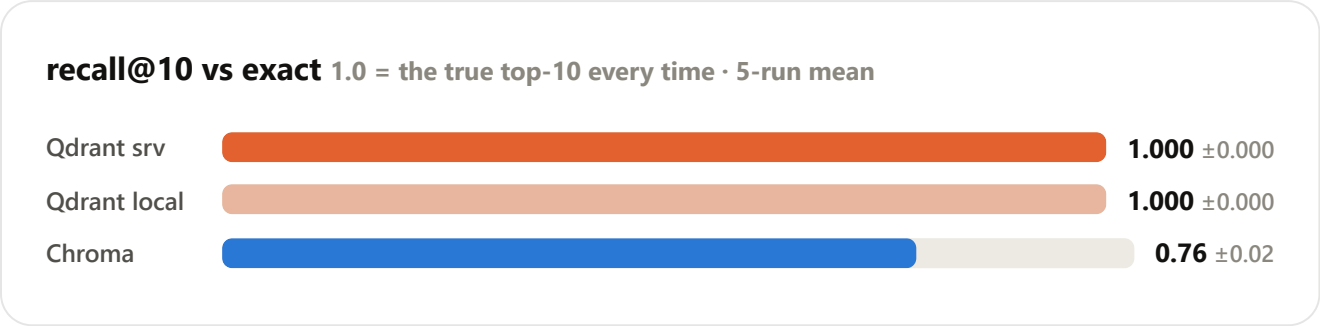
PERCENTILE (MS)	CHROMA	QDRANT LOCAL	QDRANT SERVER
p50 (median)	**1.39**	32.3	15.4
p90	**1.68**	34.8	24.8
p95	**1.72**	36.3	26.5
p99	**1.99**	40.1	30.0
Filtered (doc_type) mean	**11.9**	93.3	27.0
Filtered p95	**13.1**	99.6	32.7

Note the filtered case: the Qdrant server drops from the local mode's 93 ms to 27 ms — its payload indexes make metadata filtering cheap. Chroma still leads on absolute latency, but the server's filter penalty (+12 ms over its own unfiltered) is proportionally the smallest of the three.

07 — RETRIEVAL QUALITY

Recall — do we get the *right* results?

Recall@10 vs an exact brute-force scan. Qdrant returns the true top-10 on every run, in both modes; Chroma's default HNSW misses ~24%.



The gap is tunable, not a ceiling

Chroma's default HNSW search breadth (`ef`) favours speed, exploring fewer candidates. Raising `ef_search` trades some of its large latency surplus for recall. The 0.76 is the out-of-the-box figure. Across 5 runs it drifts 0.73–0.78 (HNSW build is randomised); Qdrant is deterministic at 1.000.

08 — ANSWER QUALITY

Same question, same answer — the finding that matters most

What an institution ships is the *answer*. We ran the full RAG pipeline (retrieve → Gemini) on identical questions through each backend and compared the generated text.

CROSS-DB RETRIEVAL AGREEMENT	VALUE	READING
top-10 overlap	0.76	tracks Chroma's recall — the tail differs
top-1 agreement	0.68	same #1 result on ~68% of queries
score MAE (shared hits)	1.8×10^{-7}	identical scoring — same vectors, same cosine
end-to-end answer similarity	**0.98**	the generated answers are effectively the same

0.98 mean answer similarity across the tested rent-deed questions (0.979, 0.986, 0.979) — **1.0 = effectively identical**.

Despite a 0.76 top-10 overlap, the answers converge because the **decisive, high-signal chunks** (the relevant statute, the "liquidated damages" definition, the matching red-

flags) are retrieved by *both* stores. For the questions below, the two backends retrieved the **identical top-9 sources with identical scores** — so the answers differ only in Gemini's phrasing.

Example — "Is the non-refundable security deposit enforceable in India?"

CHROMADB · CONFIDENCE: HIGH

"No … the clause stating the security deposit is 'NON-REFUNDABLE … forfeited to the Lessor in all circumstances' is unlikely to be fully enforceable. In India, courts treat such clauses, if essentially a penalty, under the principles of liquidated damages … will only award reasonable compensation … grounded in **Section 74 of the Indian Contract Act, 1872**."

QDRANT · CONFIDENCE: HIGH

"While the Rental Deed states … the deposit is 'NON-REFUNDABLE and shall stand forfeited … including normal expiry,' its enforceability is subject to scrutiny. As per the definition of Liquidated damages (referencing **Section 74, Indian Contract Act, 1872**), courts award only reasonable compensation … and will not enforce the sum if it is a penalty."

Both grounded on the same retrieved set — Indian Contract Act §58 & §36, the "Liquidated damages" glossary entry (§74), red-flag references, two Supreme Court judgments — **identical sources, identical scores.** Similarity: 0.979.

09 — LOCAL VS SERVER

Why Qdrant's mode changes the verdict

The single most important methodological point: Qdrant's **embedded local mode is not its performance path**. Running the real Rust server reversed two headline results.

QDRANT	LOCAL (EMBEDDED)	SERVER (RUST)	CHANGE
Index build	54.9 s	**6.2 s**	**9× faster**
Delete	290 ms	**37 ms**	**8× faster**
Query mean	32.2 ms	**14.7 ms**	**2.2× faster**
Filtered query	93.3 ms	**27.0 ms**	**3.5× faster**
recall@10	1.000	1.000	unchanged

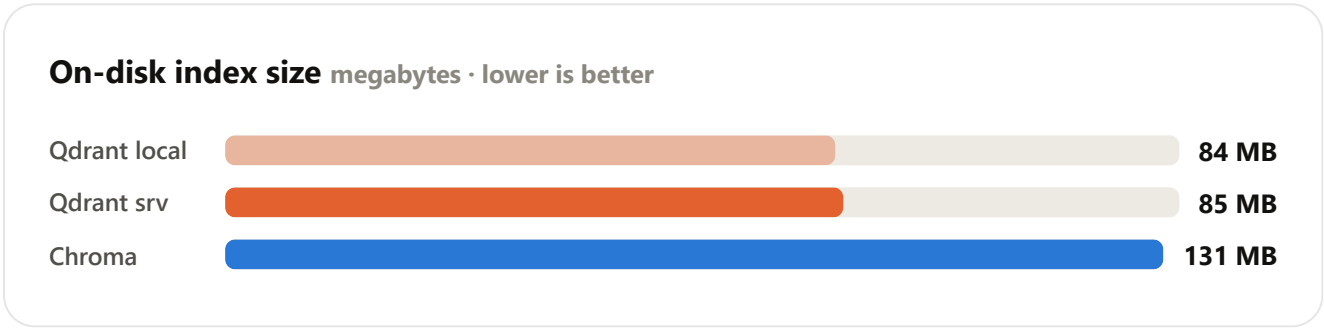
Why Chroma still wins single-query latency — and why that's expected. ChromaDB runs *in the same process* as the app: a query is a function call with zero network cost. Qdrant, even on localhost, is a *separate server* — every query pays a gRPC round-trip

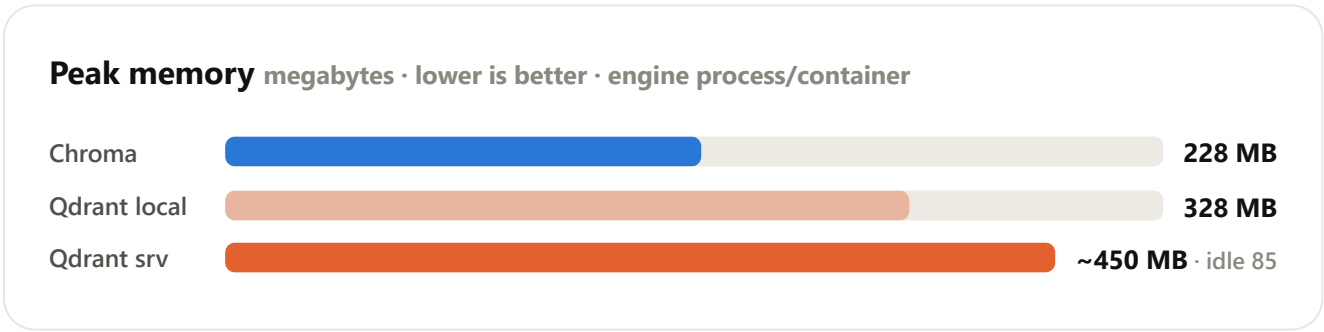
and (de)serialization (~10–15 ms here). That transport is the residual gap, not slower vector search. It's the classic **embedded vs client-server** trade-off: in-process wins a single synchronous query; a server wins on concurrency, horizontal scale, isolation, and shared multi-client access — precisely what production platforms need, and where the server's throughput advantage reasserts under load.

10 — RESOURCE USAGE

Disk & memory

Qdrant stores the same index in ~35% less disk in both modes; Chroma keeps the lowest memory footprint. The Qdrant server's ~450 MB peak is transient bulk-load memory — it idles at 85 MB.

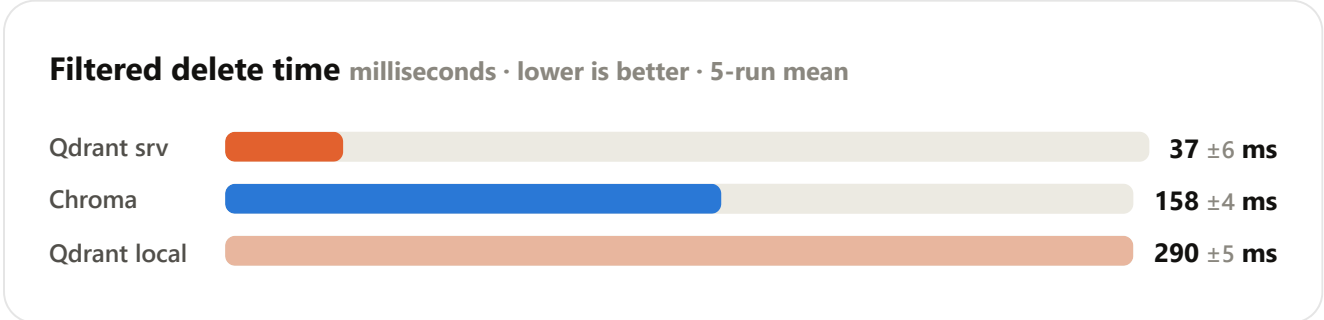




11 — MUTATION

Delete performance

Legal EASE evicts a document's chunks when it expires, so filtered-delete cost matters. The Qdrant server is dramatically fastest; its local mode is slowest.



12 — ARCHITECTURE

Operational & architectural comparison

DIMENSION	CHROMADB	QDRANT
Core language	Python (+ hnswlib)	Rust
Embedded / local mode	Yes — production-grade single node	Yes — dev convenience (
Server / distributed	Server exists; clustering newer	First-class: sharding, repl
Concurrency (one store)	Multiple in-process readers	Local locks the folder; se
Metadata filtering	`where` dict	Typed filters + payload i
Quantization / mem tuning	Limited	Scalar / product / binary,
Default recall (measured)	**0.76 (tunable up)**	**1.000**
Single-query latency (measured)	**~1.4 ms in-process**	~15 ms server (transport
Server-side index / delete (measured)	9.3 s / 158 ms	**6.2 s / 37 ms**
Managed cloud	Chroma Cloud (newer)	Qdrant Cloud (mature)
License	Apache-2.0	Apache-2.0

13 — DECISION

Which should you choose?

Choose ChromaDB when…

- ▲ Single-node, ≤ ~1M vectors; latency is critical
- ▲ You want zero server ops / fastest MVP
- ▲ Memory is tighter than disk
- ▼ …and you'll raise HNSW `ef` if recall matters

Choose Qdrant (server) when…

- ▲ Millions of vectors, HA, horizontal scale, concurrency
- ▲ Out-of-the-box recall accuracy is non-negotiable
- ▲ Fast bulk indexing & deletes, indexed filtering, quantization
- ▼ …and you'll run the server, not embedded mode

Recommendation for Legal EASE

Stay on **CHROMADB** for the current product — faster in-process, lighter, and answer quality is indistinguishable (0.98). Keep the pluggable `VectorStore` abstraction and the Qdrant backend: when the corpus outgrows a single node or recall SLAs tighten, flip `VECTOR_DB=qdrant` against a **Qdrant server** — proven here to index faster, delete faster, and return perfect recall — with no application rewrite. Before that switch, close Chroma's recall gap by raising its HNSW search breadth and re-running this benchmark.

Full per-run data

Local-mode benchmark — Chroma vs Qdrant local (5 runs)

RUN	C IDX S	C P95 MS	C RECALL	Q IDX S	Q P95 MS	Q RECALL
1	9.6	2.07	0.732	56.2	34.79	1.000
2	9.3	1.66	0.768	53.8	35.50	1.000
3	9.1	1.61	0.782	52.7	35.17	1.000
4	9.1	1.62	0.757	57.6	36.77	1.000
5	9.3	1.63	0.768	54.2	39.28	1.000
mean	**9.28**	**1.72**	**0.761**	**54.9**	**36.3**	**1.000**

Server-mode benchmark — Chroma vs Qdrant server (5-run aggregate)

METRIC	CHROMA	QDRANT SERVER
Index build (s)	9.3 ± 0.2	6.2 ± 0.3
Throughput (vec/s)	980 ± 23	1476 ± 77
Query mean (ms)	1.42 ± 0.08	14.71 ± 0.37
Query p95 (ms)	1.78 ± 0.15	26.48 ± 0.27
Filtered mean (ms)	11.65 ± 0.29	26.97 ± 1.37
recall@10	0.772 ± 0.024	1.000 ± 0.000
Delete (ms)	164.4 ± 9.3	37.0 ± 5.6
Disk (MB)	130.7	85 (container)
Peak memory (MB)	229	~450 (idle 85)

Limits. Single machine, CPU-only, 9,063 vectors — may not extrapolate to billion-scale or GPU-served deployments. Qdrant server measured on localhost (transport ~sub-ms, realistic for co-located services). Chroma recall is HNSW-tunable. Answer similarity used 3 rent-deed questions on `gemini-2.5-flash`. Machine-readable data: `docs/benchmark_data_local.json` and `docs/benchmark_data_server.json`.

Legal EASE — AI-powered Indian legal document analysis. Master study v3. Data: `scripts/benchmark_vector_dbs.py --runs 5` (local & server) and `scripts/compare_answers.py`. Embeddings: law-ai/InLegalBERT (768-d, cosine). Qdrant server: v1.18.3 (Docker). Both databases Apache-2.0.